

Function Weaver Proposal

Function Weaver: Graph-Augmented Progressive Reification for Human-AI Collaborative Software Development ##### 一、研究背景与痛点 (Problem Statement & Motivation)

当前以 Copilot / Cursor 为代表的 Agentic IDE 展现了强大的代码生成能力。但在处理中大型需求（从自然语言 PRD 到完整系统架构）时，其原生“规划（Plan）”模式面临三大系统性缺陷：

- 发散与难以收敛**：大模型对开放式自然语言的自回归推理具有高度不确定性，面对相同需求，多次生成的系统架构模块差异巨大。
- 企业级“隐性需求”黑洞**：现有的模型规划往往只聚焦于用户显式提出的“业务主线”（如文件上传），而严重遗漏权限鉴权、并发限流、全局错误兜底等软件工程中必须的“隐性基建”。
- 黑盒执行与极高的纠错成本**：现有的 Plan 是一次性生成的纯文本。由于缺乏细粒度的人类干预节点，一旦上游规划出现遗漏，将导致下游生成的数百行代码面临推翻重来的高昂成本。

核心洞察 (Key Insight)：软件工程的底层结构是高度收敛的，而大模型的生成天性是发散的。解决这一矛盾的关键，不在于训练一个更聪明的规划 Agent，而在于在 LLM 的“高层意图规划”与“底层代码执行”之间，硬性嵌入一个**带有先验领域知识的交互式边界对象 (Interactive Boundary Object)**。

二、系统架构与 workflow (System Architecture & Pipeline)

本项目提出 Function Weaver，一个基于 Model Context Protocol (MCP) 与 IDE 插件强协同构建的“混合状态机中间件”。它巧妙解决了 LLM 工具调用的超时隐患，通过以下“四步走”工作流（The Weaver Pipeline），实现了从自然语言到高精度架构树的收敛：

第一步：自然语言摸底与意图锚定 (Agent 主场)

- 场景**：用户在 IDE (如 Cursor) 中输入：“我想做一个给内部员工用的文档管理系统”。
- 行为**：原生 Agent 发挥其强大的语义理解能力，与用户确认高阶边界，提炼出核心的“根节点 (Root Nodes)”（例如：企业微信登录、文档树状目录、富文本编辑器）。

第二步：系统级强引导与工具调用 (Prompt-Driven Tool Call)

- 机制约束**：通过注入项目级系统提示词（如 `.cursorrules`），强制建立工作流边界规则：“在生成任何系统级代码前，**必须先调用 `weaver_plan_architecture` 工具生成功能图谱，并等待人类确认**”。
- 数据流**：Agent 遵循指令，将聊天中抽象出的根节点作为 JSON 参数发起 MCP 工具调用：

```
{
  "root_nodes": ["企业微信登录", "文档树状目录", "富文本编辑器"]
}
```

第三步：图谱注入、异步挂起与视觉反转 (Weaver 核心机制 / 绕过超时壁垒)

- 后台图谱注入 (MCP Server)**：本地 MCP Server 收到根节点后，查询预设的“企业级应用功能图谱”。基于规则引擎补充隐性需求（如自动挂载 `Token`刷新、并发异地登录踢出等基建节点），构建出一棵完整的强类型 JSON 树。
- 秒级返回与防超时 (Fast Return)**：为避免人类在 UI 交互时触发 LLM 严苛的 API Timeout（通常 60秒），MCP Server 并不阻塞等待，而是立即向 Agent 返回一个异步凭证：`{"status": "waiting_for_human", "message": "图谱已生成并在 UI 中打开，请等待用户确认"}`。Agent 收到此信息后自动结束当前生成轮次 (Turn)，实现优雅挂起。

- **进程间通信与前台视觉拦截 (VS Code Extension)**: 同时, MCP Server 通过本地 IPC 唤醒 Function Weaver IDE 插件, 插件调用 Webview 弹出交互式画板。
- **个性化编织 (Human-in-the-Loop)**: 用户在可视化的树状图中, 看到系统补全的数十个企业级节点。用户通过拖拽、增删、添加注释, 对功能树进行充裕的白盒修改与审阅。(**The Tree is the Prompt**)

第四步: 上下文自动注入与精准生成 (自动闭环)

- **隐式回传机制**: 用户在画板点击“确认并生成代码”。此时并非由 MCP 慢速返回结果, 而是由 IDE 插件在后台将修改后的终版 JSON Schema 打包, **自动向 IDE 的 Chat 窗口注入一条隐式的追问指令** (模拟用户发言): “我已经确认了系统架构, 这是最终的 JSON Schema: {...}, 请严格按照此结构生成代码。”
- **结果**: Agent 接收到唤醒信号和这份毫无歧义的、极度详尽的 Context 后, 像一台精密打字机一样, 极其精准地开始输出下游的高质量代码。

三、核心学术与工程贡献 (Contributions)

1. **架构层 (异步混合状态机 Asynchronous State Machine)**: 设计了一种规避 LLM 严苛超时机制的异步交互模式, 通过“MCP 管道 + IDE 视图”的职责解耦, 首次在代码生成 Agent workflows 中跑通了深度的 Human-in-the-Loop 拦截。
2. **机制层 (知识注入图谱 Graph Injection)**: 提出了一种结合“通用大模型动态推理”与“静态领域规则图谱”的混合规划架构, 大幅提升了生成代码的企业级可用性下限。
3. **交互层 (树即提示词 The Tree is the Prompt)**: 设计并验证了全新的交互范式, 将不可控的 Text-to-Code 转化为高确定性的 `Text -> Visual Tree -> Code`, 显著降低了开发者掌控复杂系统架构的认知负荷。

四、评估与实验设计 (Evaluation / User Study)

为全面评估 Function Weaver 在系统收敛性、代码质量以及人机协同体验上的有效性, 本项目将采用“混合方法研究 (Mixed-methods Research)”, 结合受控用户实验与线上真实环境的纵向日志分析。

4.1 受控用户实验 (Controlled User Study)

- **任务设计 (Tasks)**: 选取 2-3 个具有典型“隐性基建”特征的中大型系统需求。例如: “开发一个支持文件分块上传、附带用户鉴权与并发限流的 Node.js 后端服务”。
- **评估指标 (Dependent Variables)**:
 - **客观指标**: 任务完成时间 (Task Completion Time)、生成的底层基建代码覆盖率 (如是否遗漏日志记录、全局错误拦截等)、一次性编译通过率。
 - **主观指标**: 使用 NASA-TLX 量表评估开发者的认知负荷; 使用 Likert 量表评估开发者对 AI 生成架构的**控制感 (Perceived Control)** 与**信任度 (Trust)**。

4.2 线上环境部署与遥测日志分析 (In-the-Wild Deployment)

- **核心日志挖掘指标 (Telemetry Metrics)**:
 - **视觉交互深度 (Visual Interaction Depth)**: 记录开发者在可视化的“功能节点树”上的平均停留时间、对节点进行“增删改查”的平均次数。
 - **指令重试衰减率 (Prompt Iteration Reduction)**: 对比安装 Weaver 前后, 开发者发起多轮对话修正的频次变化, 验证系统是否有效降低了“纠错成本”。

五、预期结论 (Expected Outcomes)

Function Weaver 将证明：在 Agentic 软件工程中，通过引入外部图谱知识库和建立关键的“视觉交互停顿点 (Visual Interception) ”，系统能够在不损失 LLM 自动化效率的前提下，极大地提高架构规划的稳定收敛性，并确立更可靠的人机信任共识。